

Reading

Sambasivan, N., Kapania, S., Highfill, H., Akrong, D., Paritosh, P., & Aroyo, L. M. (2021, May). [“Everyone wants to do the model work, not the data work”: Data Cascades in High-Stakes AI](#). In Proc. Conference on Human Factors in Computing Systems (pp. 1-15).



Machine Learning in Production Data Quality

Administrativa

Milestone 1 due tonight

Team feedback due 24h later

Milestone 2 is a two-week assignment

Due Oct 27 (4 weeks from today) due to midterm and fall break

Don't wait until after fall break to start

Learning Goals

- Consider data quality as part of a system; design an organization that values data quality
- Distinguish precision and accuracy; understanding the better models vs more data tradeoffs
- Use schema languages to enforce data schemas
- Design and implement automated quality assurance steps that check data schema conformance and distributions
- Devise infrastructure for detecting data drift and schema violations

Poor Data Quality has Consequences

Case Study: Inventory Management



Goal: Train an ML model to predict future sales; make decisions about what to (re)stock/when/how many...

GIGO: Garbage in, garbage out

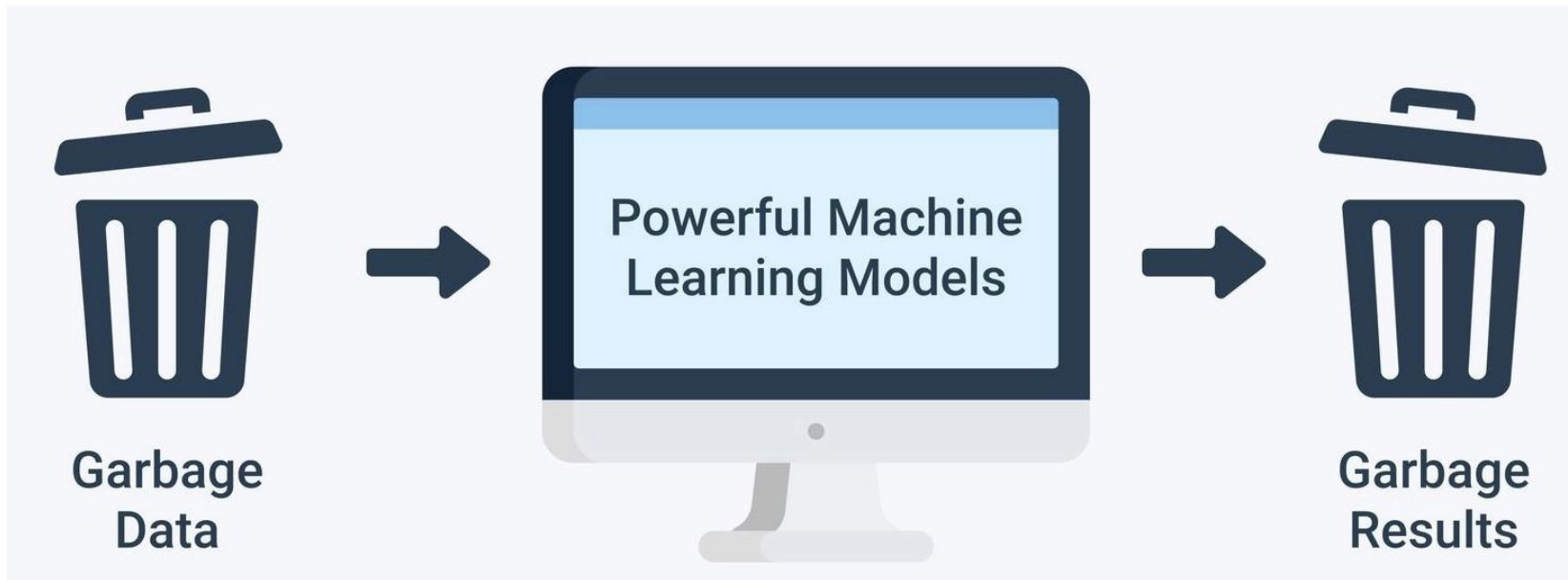


Image source: <https://monkeylearn.com/blog/data-cleaning-python>

Garbage In, Garbage Out: Target Canada

75,000 products entered manually into SAP by inexperienced staff with minimal training

No data validation rules, the system accepted anything, including nonsense, wrong units (inches vs. cm), wrong field order, missing tariff codes, vague descriptions

Data accuracy estimated at
~30% at launch
(vs. 98–99% at Target U.S.)

ML/forecasting systems were
consuming corrupt data from day
one

Canadian
Business

People Ideas Design Podcasts ☰ 🔍

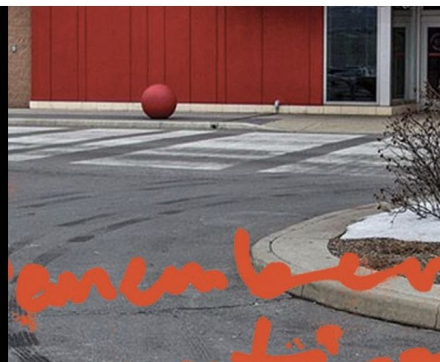
Ideas

The Last Days of Target

The untold tale of Target Canada's difficult birth, tough life and brutal death

By Joe Castaldo

Jan 01, 2016



Garbage In, Garbage Out: Target Canada

Consequences:

- Incorrect product dimensions → items couldn't fit in shipping containers → overseas shipments held at customs or delayed for weeks
- Replenishment system relied on accurate shelf/product dimensions to calculate restocking → data was wrong → system couldn't trigger reorders → shelves went empty
- Forecasting system consumed bad sales projections → ordered massive excess inventory → \$941M loss in year one, largely from markdowns on unsellable overstock
- System showed items as "in stock" when shelves were physically bare → headquarters was blind to the real situation in stores

<https://canadianbusiness.com/ideas/the-last-days-of-target-canada/>

Example Repo

Runnable code versions and extended examples of all parts of this lecture in this repo:

<https://github.com/mlip-cmu/data-quality>

Feel free to follow along and experiment with it during the lecture

Data Quality vs Quantity

Data cleaning and repairing account for about 60% of the work of data scientists.

"Everyone wants to do the model work, not the data work"

Own experience?



Accuracy vs. Precision (on Data)

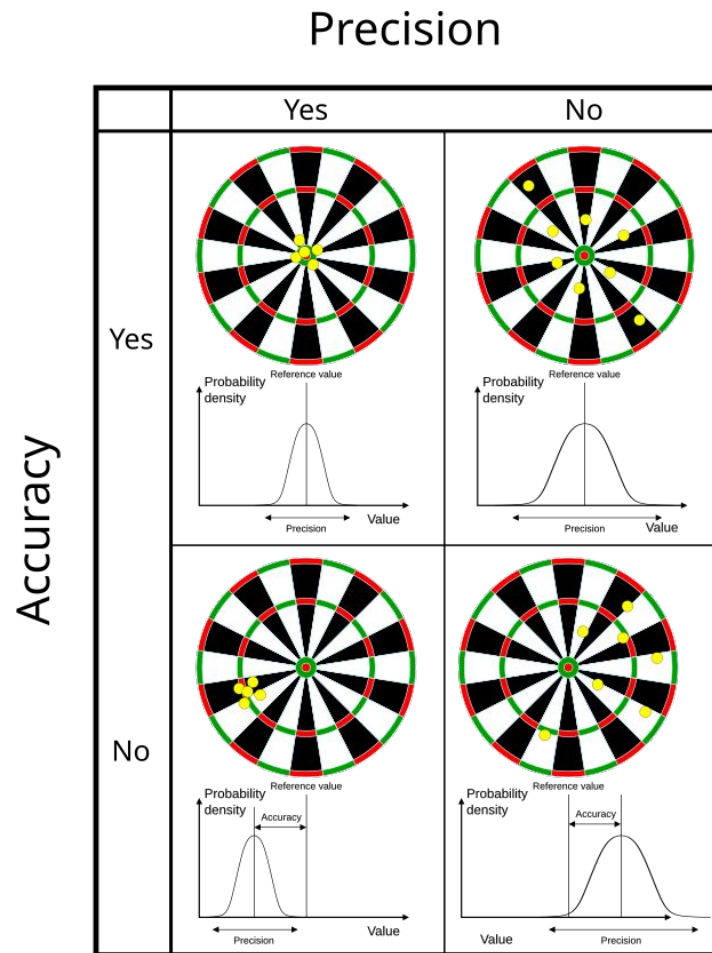
Accuracy: Reported values (on average) represent real value

Precision: Repeated measurements yield the same result

Accurate, but imprecise: **Q. How to deal with this issue?**

Inaccurate, but precise: ?

(CC-BY-4.0 by [Arbeck](#))



Data Accuracy and Precision: Impact on ML

More data → better models (up to a point, diminishing effects)

Noisy data (imprecise) → less confident models, more data needed

- some ML techniques are more or less robust to noise (more on robustness in a later lecture)

Inaccurate data → misleading models, biased models

- Need the “right” data

Invest in data quality, not just quantity

Raw Data is an Oxymoron

Shipment Received Note

Goods Received Receipt

[illegible]

Recommended Reading: Gitelman, Lisa, Virginia Jackson, Daniel Rosenberg, Travis D. Williams, Kevin R. Brine, Mary Poovey, Matthew Stanley et al. "[Data bite man: The work of sustaining a long-term study.](#)" In "Raw Data" Is an Oxymoron, (2013), MIT Press: 147-166.

What do we mean by clean data?

Accuracy: The data was recorded correctly.

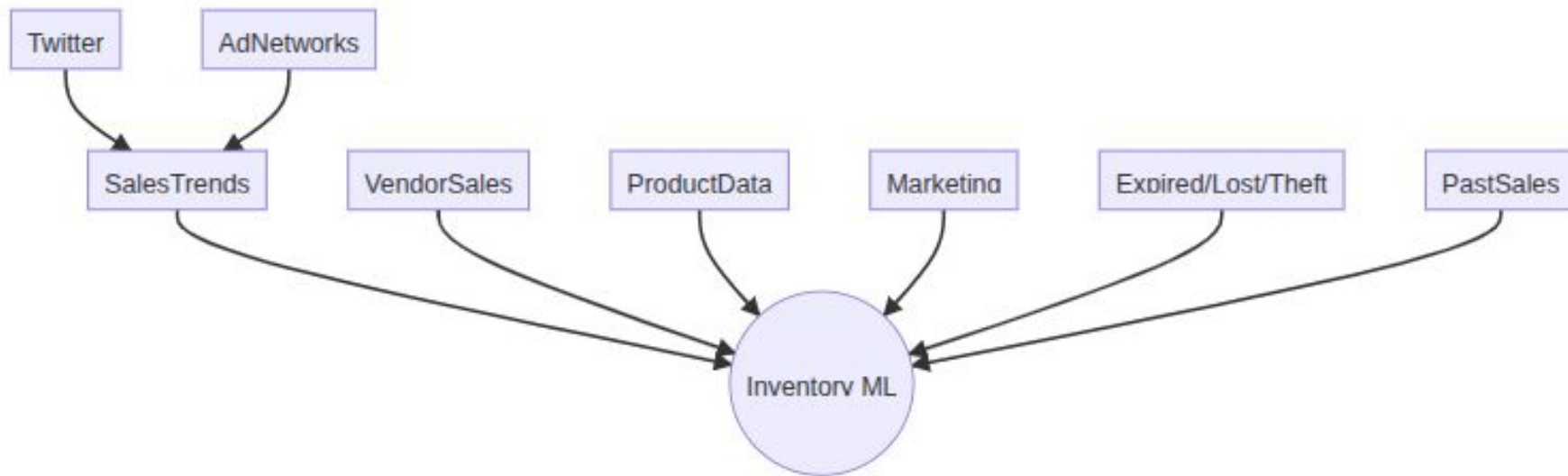
Completeness: All relevant data was recorded.

Uniqueness: The entries are recorded once.

Consistency: The data agrees with itself.

Timeliness: The data is kept up to date.

Data comes from many sources



Data comes from many sources

- Manually entered
- Generated through actions in IT systems
- Logging information, traces of user interactions
- Sensor data
- Crowdsourced

These sources have different levels of reliability and quality

What happens: Data is noisy

Wrong results and computations, crashes

Duplicate data, near-duplicate data

Out of order data

Data format invalid

Two levels of data precision

1. **Data Integrity / Schema:** Ensuring basic consistency about shape and types
2. **Wrong and inconsistent data:** Application- and domain-specific data issues

Data Integrity / Schema

Ensuring basic consistency about shape and types

Schema in Relational Databases

```
CREATE TABLE Suppliers (  
    ID INT PRIMARY KEY,  
    Name VARCHAR(255) NOT NULL,  
    ContactName VARCHAR(255),  
    ContactPhone VARCHAR(20)  
);  
  
CREATE TABLE Products (  
    ID INT PRIMARY KEY,  
    Name VARCHAR(255) NOT NULL,  
    Category VARCHAR(50),  
    UnitPrice DECIMAL(10, 2) NOT NULL,  
    QuantityInStock INT NOT NULL,  
    Unit VARCHAR(5) NOT NULL CHECK (Unit IN('count', 'kg', 'liter'))  
    SupplierID INT,  
    FOREIGN KEY (SupplierID) REFERENCES Suppliers(ID)  
);
```

Data Schema

Define the expected format of data

- expected fields and their types
- expected ranges for values
- constraints among values (within and across sources)

Data can be automatically checked against schema

Protects against change; explicit interface between components

Schema Problems: Uniqueness, data format, integrity, ...

- Illegal attribute values: `UnitPrice="Kraft Dinner 225g"`
- Violated attribute dependencies: `ImportedFrom="China"`, `TariffCode=NULL`
- Uniqueness violation: `(ID=10421, UPC="062891000017")`, `(ID=23177, UPC="062891000017")`
- Referential integrity violation: `product=(ID=10421, SupplierID=502)` if supplier 502 not defined

Dirty Data: Example

TABLE: PRODUCT

ID	Name	Category	UnitPrice	Unit	Case L×W×H (cm)	SupplierID
10421	Kraft Dinner 225g	pantry	1.29	count	40 × 30 × 20	204
10421	Bananas	produce	0.69	kg	15.7 × 11.8 × 9.8	201
10422	Tide Pods 42 ct	produce	13.99	liter	30 × 20 × 25	205
10423	MISC ITEM	pantry	Cheerios	count	999 × 999 × 999	203

TABLE: SUPPLIER

ID	Name	Phone	City	Province
201	Fresh Fields Produce	999-999-9999	Mississauga	ON
203	General Mills	416-555-0199	Toronto	Ontario
204	Kraft Heinz Canada	416-555-0123	Toronto	ON

Problems with this data? Which Problems are Schema Problems?

Data Schema Constraints for Inventory System?

Product Database:

ID	Name	Weight	Description	Size	Vendor
...	...	...	...	...	...

Stock:

ProductID	Location	Quantity
...	...	...

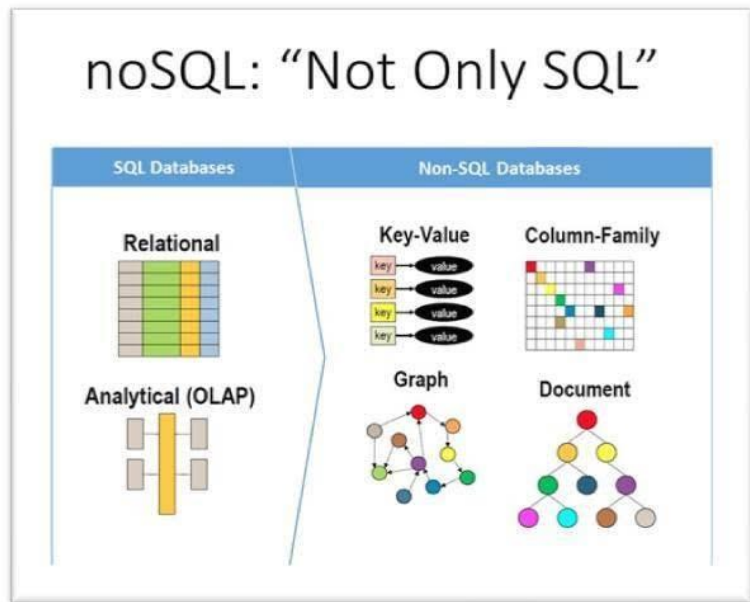
Sales history:

UserID	ProductId	DateTime	Quantity	Price
...	...	...	...	...

What Happens When New Data Violates Schema?



Modern Databases: Schema-Less



Also vector databases, schema-aware databases that basically store long text in each cell, etc.

Image source: https://www.kdnuggets.com/2021/05/nosql-know-it_all-compendium.html

Schema-Less Data Exchange

- CSV files
- Key-value stores (JSON, XML, NoSQL databases)
- Message brokers
- REST API calls
- R/Pandas Dataframes

```
ProductID,Product Name,Quantity,Unit,SupplierID  
101,Banana,50,kg,201  
102,Cucumber,75.5,kg,502  
103,Cauliflower,30,count,283
```

Schema-Less Data Exchange

Q. Benefits? Drawbacks?

Q. How to enforce a schema?

CSV Schema

```
version 1.1
@totalColumns 5
id: positiveInteger
name: length(1,255)
quantity: numericLiteral
unit: is("count") or is("kg") or is("liter")
supplierId: positiveInteger
```

<https://digital-preservation.github.io/csv-schema/>

Pydantic

```
from litellm import completion
from pydantic import BaseModel
class LineItem(BaseModel):
    product: str
    quantity: int

response = completion(
    model="gemini/gemini-2.0-flash",
    messages=[{"role": "user", "content": "Extract: 12 cases of bananas"}],
    response_format={
        "type": "json_schema",
        "json_schema": {
            "name": "line_item",
            "schema": {
                "type": "object",
                "properties": {
                    "product": {"type": "string"},
                    "quantity": {"type": "integer"}
```

Pydantic

```
from pydantic import BaseModel
```

```
messages=[  
    {"role": "system", "content": "Extract the event information."},  
    {"role": "user", "content": "Alice and Bob are going to a science fair on  
Friday."}, ]
```

```
class CalendarEvent(BaseModel):
```

```
    name: str
```

```
    date: str
```

```
    participants: list[str]
```

```
resp = completion(  
    model="...",  
    messages=messages,  
    response_format=CalendarEvent,  
)
```

Pydantic checks data against a Python class

1 • You write a class

Field names with Python types. The class is the schema.

2 • Pydantic parses data into it

JSON or a dict goes in, a typed Python object comes out. "36" becomes the number 36.0.

3 • Or it raises `ValidationError`

One entry per problem: which field, and what is wrong with it.

Pydantic is a Python library; web frameworks and LLM libraries accept its classes.

```
from pydantic import BaseModel

class LineItem(BaseModel): # simplified
    product: str
    quantity: float
    unit: str

good = '{"product": "Banana", "quantity": "36", "unit": "kg"}'
LineItem.model_validate_json(good)
→ LineItem(product='Banana', quantity=36.0, unit='kg')

bad = '{"product": "Banana", "quantity": "fifty"}'
LineItem.model_validate_json(bad)
→ ValidationError: 2 validation errors for LineItem
   quantity  Input should be a valid number
   unit      Field required
```

Our schema for a delivery e-mail: DeliveryNotice

```
class LineItem(BaseModel): # one product in the delivery
    model_config = ConfigDict(extra="forbid") # no unknown fields
    product: str
    quantity: float
    unit: Literal["count", "kg", "liter"] = Field( # one of these
        description="convert lb to kg, gallons to count")
    @model_validator(mode="after") # our own check
    def plausible(self):
        if self.quantity <= 0:
            raise ValueError("quantity must be positive")
        if self.unit == "count" and not self.quantity.is_integer():
            raise ValueError("not a whole number of items")
        return self

class DeliveryNotice(BaseModel):
    supplier: str
    delivery_date: date
    items: list[LineItem] # one LineItem per product
```

For each e-mail, the LLM fills in one

DeliveryNotice, with one **LineItem** per product.

SHAPE → JSON SCHEMA

Types, allowed units, required fields, no extra fields

Pydantic writes them out as a JSON schema, which the LLM gets with the request (response_format). Some LLM APIs enforce it, some do not.

MEANING → VALIDATORS

Positive quantities; whole numbers for count

These rules are Python code, so the JSON schema leaves them out. Pydantic checks them on every line item when it parses the answer.

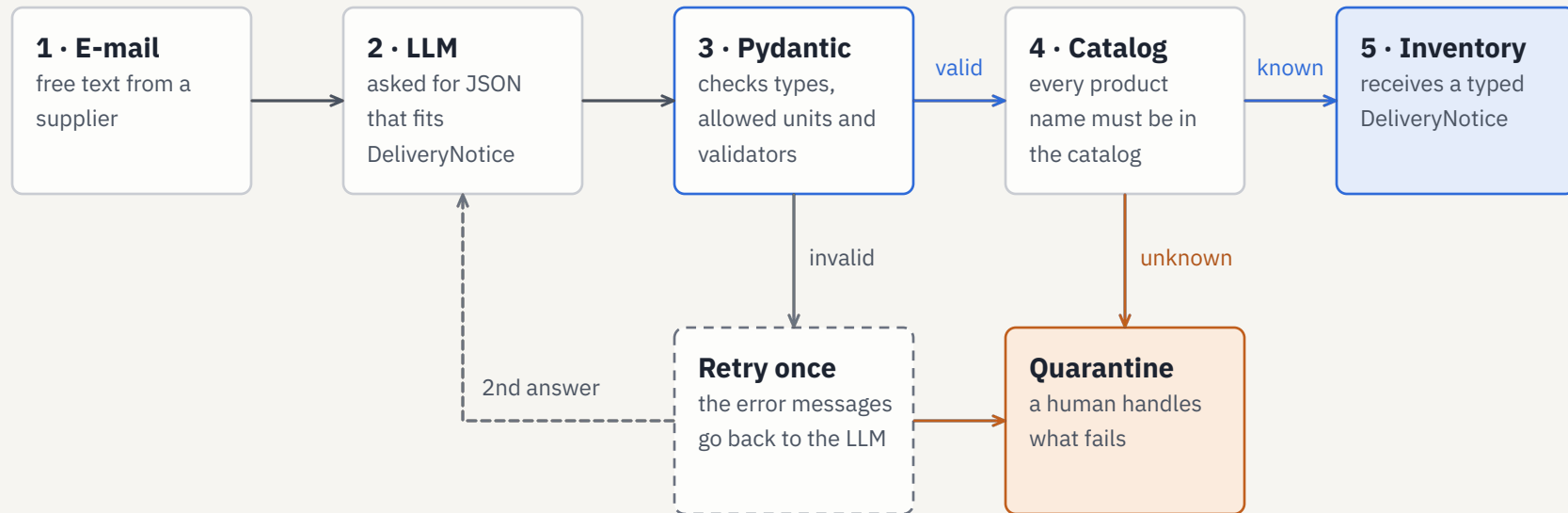
Pydantic turns the class into a JSON schema

In the LineItem class	In the JSON schema the LLM gets
<code>product: str</code>	<code>"product": {"type": "string"}</code>
<code>quantity: float</code>	<code>"quantity": {"type": "number"}</code>
<code>unit: Literal["count", "kg", "liter"]</code>	<code>"enum": ["count", "kg", "liter"]</code>
<code>Field(description="convert lb to kg, gallons to count")</code>	<code>"description": "convert lb to kg, gallons to count"</code>
<code>extra="forbid"</code>	<code>"additionalProperties": false</code>
<code>fields without a default value</code>	<code>"required": ["product", "quantity", "unit"]</code>
<code>@model_validator: quantity > 0, whole counts</code>	<code>nothing</code>

The LLM gets only the right-hand column, sent with the request. It reads the description as an instruction. It never sees the validator rules, so Pydantic checks them when the answer comes back.

Validate, retry once with the errors, then quarantine

The catalog is the store's table of 60 products. The prompt lists their names, and step 4 checks that the LLM used them.



Five supplier e-mails through the loop

The e-mail says	LLM's first answer	Pydantic says	Result
"36 kg of bananas"	Banana · 36 · kg	valid	stored
"8 cases of milk gallons (4 per case)"	Whole Milk · 32 · count	valid	stored
"50 lbs of potatoes"	Potato · 50 · lbs	items.1.unit: Input should be 'count', 'kg' or 'liter'	retry: 22.68 kg, stored
"40 liters laundry detergent, ..."	JSON cut off before the closing }	Invalid JSON: EOF while parsing an object	retry: whole JSON, stored
"half a pallet of the 24-packs"	Bottled Water · 0.5 · count	items.0: 0.5 is not a whole number of items	retry: 0.5 again, quarantined

Error paths count from 0: items.1.unit is the unit of the second line item.

Given the error message, the LLM fixed the wrong unit and the broken JSON on its second try. For **"half a pallet"**, 0.5 fits quantity: float, so the JSON schema accepts it; only our whole-number rule rejects it, so a human gets that e-mail.

The same class checks requests to our web API

```
# our server (FastAPI, a Python web framework)
app = FastAPI()

@app.post("/deliveries", status_code=201)
def post_delivery(notice: DeliveryNotice):
    received.append(notice)
    return {"accepted": len(notice.items)}
```

```
# a supplier's system sends, instead of an e-mail:
POST /deliveries
{"delivery_date": "2025-06-31",
 "items": [{"product": "Banana",
             "quantity": "fifty",
             "unit": "lbs"}],
 "note": "urgent", ...}
```

Because `post_delivery` takes a `DeliveryNotice`, FastAPI checks every request against that class before the function runs. This request is rejected: the reply has status code 422 ("your data is invalid") and lists one error per field. FastAPI also publishes the schema at `/openapi.json`, so suppliers can read the rules before they send anything.

Field	Error message
delivery_date	Input should be a valid date or datetime, day value is outside expected range
items.0.quantity	Input should be a valid number, unable to parse string as a number
items.0.unit	Input should be 'count', 'kg' or 'liter'
note	Extra inputs are not permitted

Apache Avro (e.g. for Kafka)

```
{  "type": "record",
  "namespace": "com.example",
  "name": "Customer",
  "fields": [{
    "name": "first_name",
    "type": "string",
    "doc": "First Name of Customer"
  },
  {
    "name": "age",
    "type": "int",
    "doc": "Age at the time of registration"
  }
]
```

Side benefit:
More efficient storage

Apache Avro: a schema for events sent between programs

Each checkout terminal sends one event per receipt line. The events go through Kafka, a message broker: producers (the terminals) write events to it, and consumers (inventory, forecasting, ...) read them.

1 • The schema is a JSON file

Producers and consumers agree on it. It names each field and its type.

2 • Every write is checked

The Avro writer rejects an event that does not fit the schema.

3 • The data is compact binary

A reader needs the schema to decode it, so the schema travels with the data or is kept in a registry.

```
{
  "type": "record",
  "name": "SaleEvent",
  "fields": [
    {
      "name": "event_id",
      "type": "string",
    },
    {
      "name": "event_time",
      "type": {
        "type": "long",
        "logicalType": "timestamp-millis"
      },
      // ms since 1970
    },
    {
      "name": "store_id",
      "type": "int",
    },
    {
      "name": "terminal_id",
      "type": "string",
    },
    {
      "name": "product_id",
      "type": "int",
    },
    {
      "name": "quantity",
      "type": "double",
      // a float
    },
    {
      "name": "unit",
      "type": {
        "type": "enum",
        "name": "Unit",
        "symbols": ["count", "kg", "liter"]
      },
      // one of these
      // an exact decimal with 2 digits after the point
    },
    {
      "name": "unit_price",
      "type": {
        "type": "bytes",
        "logicalType": "decimal",
        "precision": 8,
        "scale": 2
      },
      // used when an event has no promo
    },
    {
      "name": "promo",
      "type": "boolean",
      "default": false
    }
  ]
}
```

The writer rejects events that break the schema

```
event = {"event_id": "2-20250602-01295", "event_time": datetime(2025, 6, 2, 11, 2, 33),
        "store_id": 2, "terminal_id": "T1", "product_id": 128,
        "quantity": 0.584, "unit": "kg", "unit_price": Decimal("10.99"), "promo": False}
writer(file, schema, [event], validator=True, strict=True) # fastavro, a Python Avro library
```

Change to the event	What the writer says (shortened)
quantity = "fifty"	SaleEvent.quantity is <fifty> of type <class 'str'> expected double
unit = "lbs"	SaleEvent.unit is <lbs> of type <class 'str'> expected {'type': 'enum', ..., 'symbols': ['count', 'kg', 'liter']}
store_id removed	rejected, but the message only says that the whole event does not match; it does not name store_id
extra field cashier = "C25"	record contains more fields than the schema specifies: cashier

With `validator=True`, the writer checks each event as the terminal writes it, so a bad event never reaches Kafka or the consumers. `strict=True` adds the check for extra fields; without it, the writer drops unknown fields without an error.

Schema changes must keep old events readable

Avro decodes an event with two schemas: the one it was written with and the one the reader expects. It fills in defaults for fields the event lacks and skips fields the reader does not know.

V2 ADDS A FIELD WITH A DEFAULT

```
+ {"name": "loyalty_id",  
  "type": ["null", "string"],  
  "default": null}
```

v2 reader, v1 events: loyalty_id = None

v1 reader, v2 events: loyalty_id is skipped

Compatible: old and new programs keep working

V3 CHANGES A TYPE AND RENAMES A FIELD

```
"quantity": "double" → "string"  
"terminal_id" → "register_id"
```

v3 reader, v1 events:

SchemaResolutionError: Schema mismatch: double is not string

Incompatible: consumers on v3 cannot read the old events

A registry checks each new schema before use

Before a producer sends events with a new schema version, it registers the version in a schema registry (for example, Confluent Schema Registry). The registry refuses a version that old or new programs could not read.



New schema	New reads old events	Old reads new events	Registry
v2: adds an optional <code>loyalty_id</code>	yes	yes	accepts it
v3: quantity as text, <code>terminal_id</code> renamed	no	no	rejects it

An incompatible change then fails once, at the producer, when someone tries to deploy it. Without the registry it fails later, in every consumer, while events are flowing.

Side benefit: files with a schema are smaller

The same 64,560 sale events, saved in four file formats

Format	How it stores the events	Size	vs. CSV
CSV	text, one row per line, no types	5.31 MB	1×
JSON Lines	text, one object per line; field names repeated on every line	11.96 MB	2.3× larger
Avro, compressed	binary, one event after another; the schema once, at the start of the file	0.91 MB	5.8× smaller
Parquet, compressed	binary, one column after another; the schema at the end of the file	0.27 MB	20× smaller

Avro stores rows. Good for a stream of events that arrive and are written one at a time.

Parquet stores columns. Good for analysis: a query reads only the columns it needs, and a column of similar values compresses well.

The schema in the file keeps the types

The same events, read back into pandas from each file

	CSV	JSON Lines	Avro	Parquet
event_time	text	timestamp	timestamp	timestamp
unit_price	float	float	exact decimal	float
types come from	pandas guessing from the text	pandas guessing; dates only for names like *_time	the schema in the file	the schema in the file

CSV returns the timestamps as text, and the consumer must know to parse them. Avro returns the price as an exact decimal, as its schema says; a float cannot hold 10.99 exactly. The Parquet file has a float because it was written from a pandas float column.

Schema Checks after Loading

Custom code or libraries, e.g., *Great Expectations*:

```
expect_column_values_to_be_of_type("id", "int")
expect_column_values_to_be_unique("id")
expect_column_values_to_be_of_type("quantity", "float")
expect_column_values_to_be_between("quantity", min_value=0, max_value=None)
expect_column_values_to_be_in_set("unit", ["count", "kg", "liter"])
expect_column_values_to_be_of_type("supplierId", "int")
```

Check a table's schema after loading it, before using it

Store clerks type every delivery into the store system. Each month, about 8,000 delivery rows are loaded into pandas for inventory and forecasting. Nothing checked the rows when the clerks typed them in.

delivery_id	delivery_date	supplier_name	product_name	category	quantity	unit	entered_at
612959	2025-03-01	Three Rivers Beverages	Cola 6 pk	beverages	16	count	2025-03-01 13:33
616133	2025-03-14	Fresh Fields Produce	Banana	produce	54	kg	2025-03-14 13:20
618049	2025-03-21	Ohio Valley Farms	Potato	produce	20	kg	2025-03-21 13:01

Great Expectations (GX)

A Python library. You write a list of expectations, one testable rule each. GX runs the list on every new batch of rows and reports which rules fail, and for which values.

Pandera

A Python library. You write the table's schema as a class, like a Pydantic class, but each field is a column of a pandas DataFrame.

Each expectation in GX is one testable rule

Ten of the 14 expectations for the delivery table (arguments shortened). Rows 1–6 check each value, rows 7–8 compare columns or rows, rows 9–10 check a statistic of a whole column.

Expectation	What it says
<code>ExpectColumnValuesToBeOfType("quantity", "float64")</code>	quantity is a floating-point number
<code>ExpectColumnValuesToBeUnique("delivery_id")</code>	no two rows share an ID
<code>ExpectColumnValuesToBeBetween("quantity", min_value=0)</code>	no negative quantities
<code>ExpectColumnValuesToBeInSet("unit", ["count", "kg", "liter"])</code>	only the three units
<code>ExpectColumnValuesToBeInSet("category", known_categories)</code>	only categories from the product list
<code>ExpectColumnValuesToBeBetween("delivery_date", 2024-01-01, 2025-12-31)</code>	delivery dates in 2024 or 2025
<code>ExpectColumnPairValuesAToBeGreaterThanB("entered_at", "received_at")</code>	typed in after the delivery arrived
<code>ExpectCompoundColumnsToBeUnique([date, store, product, quantity])</code>	no delivery is in the table twice
<code>ExpectColumnMaxToBeBetween("quantity", max_value=2000)</code>	no delivery is larger than 2,000
<code>ExpectColumnMedianToBeBetween("quantity", 10, 60)</code>	a typical delivery is 10 to 60 units

The March deliveries fail 7 of 14 expectations

The March deliveries as the clerks typed them in. A month without typing mistakes meets all 14 expectations.

Expectation	Result	What GX shows
quantity ≥ 0	4 rows fail	-12, -24, -72
largest quantity $\leq 2,000$	largest is 80,000	only the largest value: 80,000 kg of bananas (the real delivery was 54 kg)
standard deviation of quantity ≤ 200	it is 1,687	only the number, not the rows that cause it
unit in count, kg, liter	46 rows fail	ct, each, Count, kilogram, kgs, Kg, KG
category from the product list	13 rows fail	Allegheny Bakery, CleanHome Supplies: supplier names in the category column
delivery date in 2024 or 2025	2 rows fail	1900-01-01, a placeholder date
no delivery in the table twice	52 rows fail	26 deliveries, each typed in twice

The per-row checks list the bad values. The statistics checks report one number each: 12 deliveries are 1,000 times too large, and GX shows only the largest.

A unique ID does not make deliveries unique

Two rows from the March deliveries

Column	Row 1	Row 2
delivery_id	612959	900247
delivery_date	2025-03-01	2025-03-01
store_id	1	1
product_name	Cola 6 pk	Cola 6 pk
quantity	16 count	16 count
received_at	2025-03-01 13:00	2025-03-01 13:00
entered_at	2025-03-01 13:33	2025-03-01 15:13

The system gives every typed-in row a new ID, so the ID check passes. The same delivery was typed in twice, 100 minutes apart. Only a check on the columns that describe a delivery finds it: ExpectCompoundColumnsToBeUnique.

Pandera: a table's schema as a Python class

The schema of the product master data: one row per product the store sells, 64 rows, read from a CSV file as text

```
class ProductMaster(pa.DataFrameModel):
    id: Series[int] = pa.Field(unique=True, gt=100)
    gtin: Series[str] = pa.Field(str_matches=r"^\d{13}$", unique=True) # GTIN: the 13-digit barcode number
    unit: Series[str] = pa.Field(isin=["count", "kg", "liter"])
    unit_price: Series[float] = pa.Field(gt=0, le=500)
    supplier_id: Series[int] = pa.Field(isin=SUPPLIERS) # the known supplier IDs
    shelf_life_days: Series[int] = pa.Field(ge=1, le=1825) # at most 5 years
    ... # 4 more columns

    @pa.check("gtin") # our own rule for one column: the barcode's last digit must fit the others
    def check_digit(cls, gtin):
        return gtin.map(gtin_is_valid)

    @pa.dataframe_check # our own rule across columns: produce is sold by count or kg
    def produce_unit(cls, df):
        return (df.category != "produce") | df.unit.isin(["count", "kg"])

class Config:
    coerce = True # first convert the text to the column types
```

Bad rows go to quarantine, good rows go on

ProductMaster.validate(data, lazy=True) collects every failure instead of stopping at the first. The failing rows go to quarantine.csv with the reason; the other rows go on to the next step.

id	Product	Value in the data	Rule it breaks
106	Tomato	unit "lbs"	allowed units; produce by count or kg
124	Whole Wheat Bread	GTIN 0702030001240	the barcode's check digit
139	Marinara Sauce 24 oz	price "2,99"	price must convert to a number
131	Frozen Shrimp 1 lb	supplier 502	known supplier IDs only
152	Dish Soap 20 oz	shelf life 9999 days	at most 1,825 days
120	Cheddar Cheese 8 oz	price -3.99	price above 0
150	Paper Towels 6 rolls, 8 rolls	two products with id 150	id and GTIN unique

Of 64 rows, 47 are valid and 17 are in quarantine; the table shows 7. **Not caught:** case sizes in inches instead of centimeters, a price that is too low, a product entered twice under a slightly different name. Each of these values is plausible on its own.

A schema can also be learned from past data

pa.infer_schema reads the February deliveries, a month without typing mistakes, and writes down the types and ranges it sees:

```
quantity: float64, greater_than_or_equal_to(2.0), less_than_or_equal_to(126.0)
store_id: int64, greater_than_or_equal_to(1.0), less_than_or_equal_to(8.0)
unit: string
```

Checking the March deliveries against this learned schema:

23

March deliveries flagged

21

are real typing mistakes

2

are correct deliveries, only larger than
any in February

The upper bound of 126 is February's largest delivery. Review a learned schema, and widen its bounds, before you use it.

Many Schema Libraries/Formats

Examples

- Avro
- XML Schema
- Protobuf
- Thrift
- Parquet
- ORC

Summary: Schema

Basic structure and type definition of data

Well supported in databases and many tools

Very low bar for data quality

Dirty Data: Breakout

TABLE: DELIVERY

ID	StoreID	Product	Received	Qty	Unit
50117	3101	Bananas	2013-03-04	36	kg
50118	3101	Bananas	2013-03-04	36	kg
50119	3109	Kraft Dinner 225g	2013-03-05	48	count
50120	3102	Cheerios 525g	2013-02-30	24	count
50121	3102	Bananas	2013-03-06	80000	kg
50122	3103	Tide Pods 42 ct	2013-03-06	12	cases

TABLE: STORE

StoreID	City	Province
3101	Toronto	ON
3102	Calgary	ON
3103	Ottawa	ON

As a group, post and follow up in #lecture, tagging group members and @mlipbot: Identify 2 data quality problems that would not be possible with schema enforcement.

Wrong and Inconsistent Data

Application- and domain-specific data issues

Dirty Data: Example

TABLE: PRODUCT

ID	Name	Category	UnitPrice	Unit	Case L×W×H (cm)	SupplierID
10421	Kraft Dinner 225g	pantry	1.29	count	40 × 30 × 20	204
10421	Bananas	produce	0.69	kg	15.7 × 11.8 × 9.8	201
10422	Tide Pods 42 ct	produce	13.99	liter	30 × 20 × 25	205
10423	MISC ITEM	pantry	Cheerios	count	999 × 999 × 999	203

TABLE: SUPPLIER

ID	Name	Phone	City	Province
201	Fresh Fields Produce	999-999-9999	Mississauga	ON
203	General Mills	416-555-0199	Toronto	Ontario
204	Kraft Heinz Canada	416-555-0123	Toronto	ON

Problems with this data? Which Problems are Schema Problems?

Wrong and Inconsistent Data

- Missing values: SupplierPhone=999-999-9999
- Misspellings: City=Torronto
- Misfielded values: Category=Kraft Heinz
- Duplicate records: Name=Kraft Dinner 225g, Name=KD Mac & Cheese 225g
- Wrong reference: delivery=(ProductID=10421, Name="Bananas") if product 10421 exists but is Kraft Dinner

Q. How can we detect and fix these problems?

How mistakes get into the data

Most mistakes happen when people type data in.

Mistake	Examples in the data
typo	"Atlantic Salmon", "oCrn Flakes 18 oz", the city "Pittsburg"
several spellings	the unit kg as Kg, kgs, KG or kilogram; count as ct, each or Count
a wrong but valid ID	a delivery of olive oil typed in with the product ID of oat milk
a value in the wrong field	the city "USA"; a supplier's name in the category field
a quantity off by a factor	grams typed as kilograms (54,000 kg of bananas); an extra digit (300 instead of 30)
a placeholder for "unknown"	the date 1900-01-01, the phone number 999-9999999, a shelf life of 9999 days
typed in twice, or never	the same cola delivery twice, 100 minutes apart; a delivery nobody typed in
swapped digits	two digits swapped in a manual stock count
an outdated price	the checkout system charges an old price instead of the catalog price

Data Cleaning Overview

Data analysis / Error detection

- Usually focused on specific kind of problems, e.g., duplication, typos, missing values, distribution shift
- Detection in input data vs detection in later stages (more context)

Error repair

- Repair data vs repair rules, one at a time or holistic
- Data transformation or mapping
- Automated vs human guided

Example Tool: Great Expectations

 great_expectations [Home](#) / [taxi.demo](#) / 20200819T024609.241003Z / 2020-08-19T02:46:09.241003+00:00 / cbb8bd044ccaa28d4db5e3d59c0be748

Actions

Validation Filter:
[Show All](#) [Failed Only](#)
[How to Edit This Suite](#)
[Show Walkthrough](#)

Table of Contents

[Overview](#)
[Table-Level Expectations](#)
[passenger_count](#)

dropdown_location_id, payment_type, fare_amount, extra, mta_tax, tip_amount, tolls_amount, improvement_surcharge, total_amount, congestion_surcharge

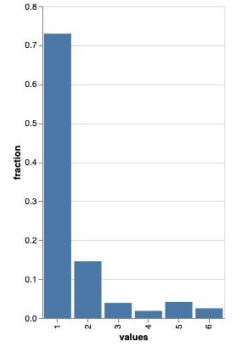
payment_type, fare_amount, extra, mta_tax, tip_amount, tolls_amount, improvement_surcharge, total_amount, congestion_surcharge

passenger_count

Status	Expectation	Observed Value
✓	values must never be null.	100% not null
✓	distinct values must belong to this set: 1.0 2.0 3.0 4.0 5.0 6.0.	[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]

✓

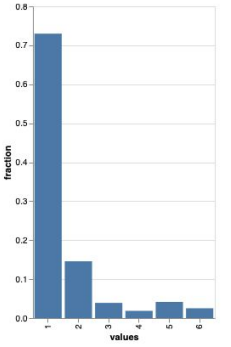
Kullback-Leibler (KL) divergence with respect to the following distribution must be lower than 0.6.



fraction

values

KL Divergence: None (-infinity, infinity, or NaN)



fraction

values

Rule-based detection: Data Quality Rules

Rules can be used to reject data or repair it

Invariants on data that must hold

Typically about relationships of multiple attributes or data sources, eg.

- ZIP code and city name should correspond
- User ID should refer to existing user
- SSN should be unique
- For two people in the same state, the person with the lower income should not have the higher tax rate

Classic integrity constraints in databases or conditional constraints

A flagged row: which value is wrong?

977 deliveries have a product name that is not the catalog name of their product ID. A check finds them all. To repair a row, we have to decide which of the two values is wrong, and what the right value is.

Delivery	Product ID says	Name typed in	The real mistake
500646	130 Atlantic Salmon	Atlanic Salmon	the name, a typo (585 deliveries like this)
500177	157 Refill Oat Milk	Refill Olive Oil	the ID: it should be 156, Refill Olive Oil (392 deliveries like this)

The next three slides repair the same 977 deliveries with three methods. They differ in the information they use to identify the wrong value:

Repair rule

a person decides in advance which value to trust

Fuzzy matching

a reference list: the product catalog

Probabilistic repair

the data itself

A repair rule always overwrites the same column

Repair rule: when the name and the product ID do not match, replace the name with the catalog name of the ID.

585

repaired correctly: every typo in a name

392

repaired wrongly: where the ID was the mistake, the correct name "Refill Olive Oil" becomes "Refill Oat Milk". The row now looks consistent, so no check flags it again.

Same failure: "take the city from the ZIP code" fixes "Pittsburg" and "USA", but moves the Monroeville store to Pittsburgh, because its ZIP code was the wrong value.

Where a rule works: mapping unit spellings (Kg, kgs, ct, each, ...) to kg, count or liter repaired 990 cells, all correctly. Each spelling can mean only one unit.

Fuzzy matching repairs only the clear cases

A string-similarity library (rapidfuzz) finds the catalog name closest to the typed name. The repair is automatic only if that name is the catalog name of the delivery's product ID and is at least 85% similar; otherwise a person decides.

571

repaired correctly, e.g. "Atlantic Salmon"
becomes "Atlantic Salmon"

0

repaired wrongly

406

sent to a person: all 392 wrong IDs,
where the typed name is exactly
another product's name, and 14 typos
that were not similar enough

Less likely to make wrong repairs because it repairs only when the typed name closely matches the catalog name of the ID. The wrong IDs go to a person. Its repairs are only as correct as the catalog.

Learning from the data which value is wrong

A small version of HoloClean, a research tool. Combinations of product ID, name, category, unit and supplier that occur in at least 30 deliveries count as correct. A rarer combination gets candidate repairs, the common combinations that differ in one or two columns, each with a probability. Repairs with a probability of at least 0.9 are applied; the rest go to a person.

Delivery as typed in	Repair	Result
ID 101 (Banana), name "Hummus 10 oz", category dairy, supplier 212	ID → 160, Hummus 10 oz	correct: category and supplier belong to hummus
ID 134 (Cola 12 pk), name "Cola 6 pk", category beverages, supplier 206	name → "Cola 12 pk"	wrong: the ID should be 133; both packs have the same category, unit and supplier

744 repaired correctly: all 585 typos and 159 wrong IDs

71 repaired wrongly, each with a probability of at least 0.9

162 sent to a person

It needs no catalog, and it is the only method that fixes wrong IDs. It works only if most rows are correct, and it fails when two products have the same values in the other columns.

Three repairs of the same 977 deliveries

Method	Knowledge comes from	Repaired correctly	Repaired wrongly	Sent to a person
repair rule	a person, once, for all rows	585	392	0
fuzzy matching	the product catalog	571	0	406
probabilistic repair	the other columns in the data	744	71	162

Each method trades wrong repairs against work for people. The rule sends no rows to a person and repairs 392 wrongly; fuzzy matching repairs none wrongly and sends 406 to a person. Probabilistic repair fixes the most rows, but its 71 wrong repairs have probabilities as high as its correct ones, so nothing separates them.

Most approaches use ML to detect inconsistencies, writing rules is hard

- Anomaly detection (e.g., 80,000 kg shipment of bananas)
- Learning common patterns and outliers (e.g., product identifier and product name mapping)
- Drift detection (e.g., average scale readings rising)
- Near duplicate detection (e.g., two identical/similar shipments on the same day)
- Learn patterns from past repairs (e.g., corrected weight in sales)

Usually identifying *suspicious data* rather than clear mistakes

ML-based for Detecting Inconsistencies

	DBAName	AKAName	Address	City	State	Zip
t1	John Veliotis Sr.	Johnnyo's	3465 S Morgan ST	Chicago	IL	60608
t2	John Veliotis Sr.	Johnnyo's	3465 S Morgan ST	Chicago	IL	60609
t3	John Veliotis Sr.	Johnnyo's	3465 S Morgan ST	Chicago	IL	60609
t4	Johnnyo's	Johnnyo's	3465 S Morgan ST	Cicago	IL	60608

Conflicts

Does not obey data distribution

Conflict

Not many standard tools, often custom data sciency solutions

Anomaly detection with
PyOD or Isolation Forest,
LOF etc. in *scikit-learn*

Near duplicate with Zingg,
Dedupe, Splink

Many academic tools, e.g.
ActiveClean, Raha,
HoloClean

Great Expectations library:

```
expect_column_values_to_be_between(  
    column="passenger_count",  
    min_value=1,  
    max_value=6  
)
```

Discussion: Data Quality Rules?



Product Database:

ID	Name	Weight	Description	Size	Vendor
...	...	...	...	...	...

Stock:

ProductID	Location	Quantity
...	...	...

Sales history:

UserID	ProductId	DateTime	Quantity	Price
...	...	...	...	...

ML ranks suspicious rows for a person to check

A model can learn what normal data looks like from the data, and rank the rows that look different. A person checks the top of the ranking; some of those rows turn out to be correct.

Approach	What it learns	Example it finds in the supermarket data
Anomaly detection	the usual values	a delivery of 40,000 kg of onions
Common patterns	which values go together	product 144 with the name "oCrn Flakes 18 oz"
Near-duplicate detection	when two rows describe one event	the same yogurt delivery typed in twice, as 24 and 26.4 tubs
Learning from past repairs	which rows people corrected before	checkout lines from a scale that weighs 15% too heavy

How we measure a detector: take the rows it ranks highest and count how many are real mistakes. That share is its precision.

An unusual quantity depends on the product

An anomaly detector gives every delivery a score for how different it is from the others, and a person checks the highest scores. 495 of the 198,815 deliveries have a wrong quantity: an extra digit, grams typed as kilograms, or a minus sign.

Features: quantity, product and weekday

100 lemons is a large number next to 5 kg of salmon, although both are normal deliveries. Many correct deliveries get high scores.

61%

of the 495 highest-scored deliveries are real mistakes

Added: the usual quantity for this store and product

Store 5 usually receives 40 kg of yellow onions per delivery, so 40,000 kg is 1,000 times the usual amount, and 100 lemons is a usual delivery of lemons.

82%

of the 495 highest-scored deliveries are real mistakes

The numbers are for IsolationForest (scikit-learn). With ECOD, another common detector, the added feature raises precision from 56% to 82%. Switching detectors changes precision by a few points; the added feature raises it by more than 20.

Learning which values go together

All deliveries with product ID 144, grouped by the name the clerk typed in:

Name typed in	Deliveries
Corn Flakes 18 oz	3,232
Tomato Soup	7
12 misspellings: Cor Flakes 18 oz, oCrn Flakes 18 oz, Corn Flkaes 18 oz, ...	12

Almost every delivery of product 144 has the same name. From this, the method learns the pattern **product ID → name** and flags the 19 deliveries that break it. Nobody wrote this rule.

The method tries every pair of columns and keeps the pairs that hold in at least 95% of rows. Most are real rules, such as "a store has one city". "A supplier delivers one category" holds in 98.7% of rows and is only a habit: the seafood supplier also sells frozen shrimp, so most rows that break it are correct.

Same 977 rows, found without writing a rule

In the repair part, a person wrote the rule "the name must be the catalog name of the product ID" and needed the catalog to check it. The learned pattern product ID → name flags exactly the same 977 deliveries, using only the delivery data.

Solved: finding the rows

Nobody has to think of the rule, and no catalog is needed.

Still open: which value is wrong

585 rows have a typo in the name and 392 have a wrong product ID. The pattern flags both kinds in the same way.

Delivery	product_id	Name typed in	Usual name for this ID	The real mistake
500430	144	oCrn Flakes 18 oz	Corn Flakes 18 oz	typo in the name
500177	157	Refill Olive Oil	Refill Oat Milk	wrong ID: should be 156

Deciding which value to change is the repair problem from the previous part, with the same three options: a repair rule, fuzzy matching or probabilistic repair.

Finding a delivery that was typed in twice

January to March 2025: 22,573 deliveries; 108 were typed in twice, some with a slightly different quantity the second time.

Delivery	Store	Product	Date	Quantity
611803	1	Greek Yogurt 32 oz	2025-02-24	24
900967	1	Greek Yogurt 32 oz	2025-02-24	26.4

Same store, product and date: strong evidence of a duplicate. Quantities 10% apart: some evidence. **Splink**, a Python library for finding records that describe the same thing, adds up this evidence field by field and gives each pair of rows a match probability; for this pair it is 0.36.

Method	Duplicates found, of 108	Pairs flagged wrongly
exact match on store, product, date and quantity	69	5
Splink, pairs with a match probability of at least 0.1	108	5

Splink learns from the data how much each matching field counts. Learned from the data alone, the weights were wrong; we set by hand that a real duplicate almost always has the same store and product.

Past repairs teach a model what to check next

At the checkout, fruit and meat are weighed; staff later correct some of these lines, and each correction is recorded. A gradient-boosting classifier learns from 20 days of past corrections which lines get corrected, using store, terminal, cashier, hour, product, quantity and price.

Next 10 days: 7,210 lines, 480 need a correction (6.7%). Staff check the 360 lines the model ranks highest (5%):

351 of 480

lines that need a correction are among them;
98% of the checked lines need one.

The causes behind the past corrections

Cause	Corrections
the scale at store 2, terminal T3, weighs 15% too heavy from March 10 on	418
the container's weight was not subtracted (meat counter)	177
a look-alike product code, e.g. banana for organic banana, often by cashier C25	78

Terminal and store contribute most to the predictions, then product: the features that match these causes.

The model only finds the kinds of mistakes that staff corrected before.

Dirty Data: Breakout 2

TABLE: DELIVERY

ID	StoreID	Product	Received	Qty	Unit
50117	3101	Bananas	2013-03-04	36	kg
50118	3101	Bananas	2013-03-04	36	kg
50119	3109	Kraft Dinner 225g	2013-03-05	48	count
50120	3102	Cheerios 525g	2013-02-30	24	count
50121	3102	Bananas	2013-03-06	80000	kg
50122	3103	Tide Pods 42 ct	2013-03-06	12	cases

TABLE: STORE

StoreID	City	Province
3101	Toronto	ON
3102	Calgary	ON
3103	Ottawa	ON

As a group, post and follow up in #lecture, tagging group members and @mlipbot: Identify 2 data quality problems that schema enforcement would not catch and suggest how to detect/fix it.

[Problem 1]: [How to detect/fix]

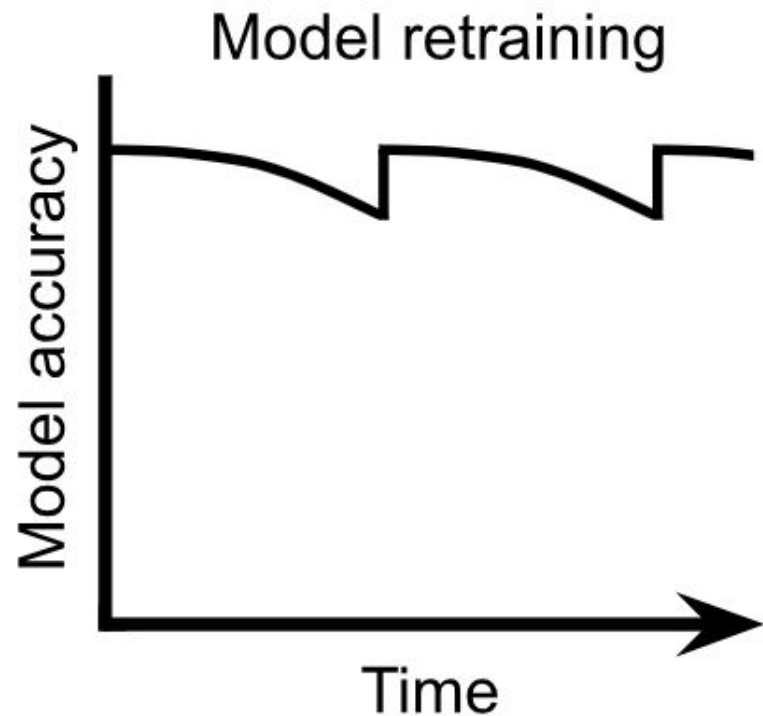
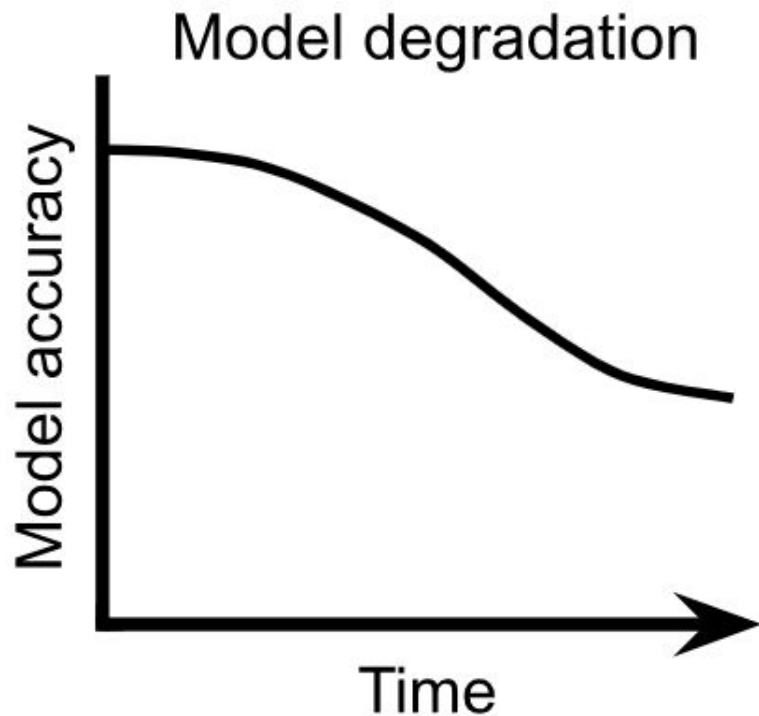
[Problem 2]: [How to detect/fix]

Dealing with Drift

Why does my model begin to perform poorly over time?

A very particular form of data accuracy problem (data becomes wrong), caused not by human creators but by the world. Very prevalent & affects product!

Models often Degrade over Time



Data changes

System objective changes over time

Software components are upgraded or replaced

Prediction models change

Quality of supplied data changes

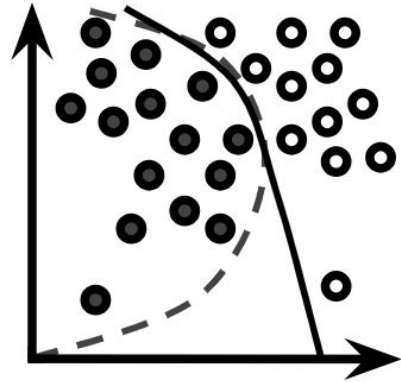
User behavior changes

Assumptions about the environment no longer hold

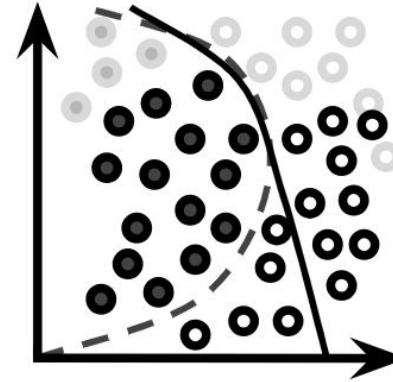
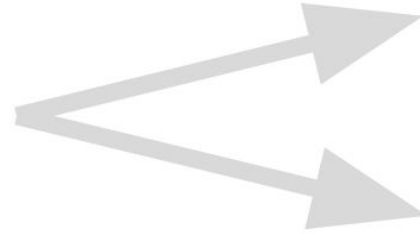
Users can react to model output; or try to game/deceive the model

Examples in inventory system?

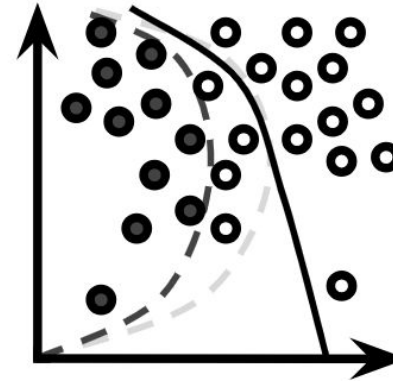
Data vs Concept Drift



Data drift:
changed distribution,
stable decision boundary,
inaccurate predictions



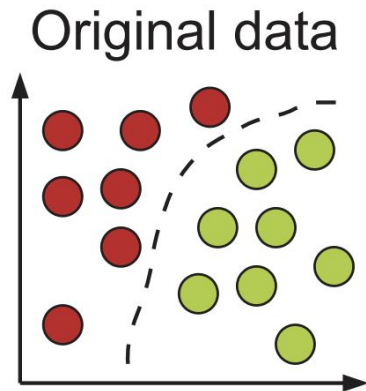
Concept drift:
changed decision boundary
stable distribution
inaccurate predictions



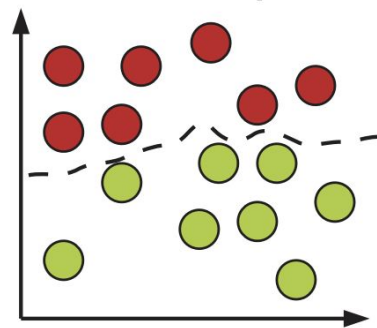
Legend:

- Data group A
- Data group B
- - - Real-world decision boundary
- Model's learned decision boundary

Types of Drift

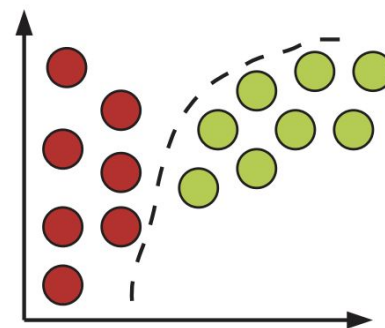


Real concept drift



$p(y|X)$ changes

Virtual drift



$p(X)$ changes, but not $p(y|X)$

Drift & Model Decay

Concept drift (or concept shift)

- properties to predict change over time (e.g., what is credit card fraud)
- model has not learned the relevant concepts
- over time: different expected outputs for same inputs

Data drift (or covariate shift, virtual drift, distribution shift, or population drift)

- characteristics of input data changes (e.g., customers with face masks)
- input data differs from training data
- over time: predictions less confident, further from training data

Schema drift (upstream data changes)

- external changes in data pipeline (e.g., format changes in weather service)
- model interprets input data incorrectly
- over time: abrupt changes due to faulty inputs

How do we fix these drifts?

On Terminology

Concept and data drift are separate concepts

In practice and literature, not always clearly distinguished

Colloquially encompasses all forms of model degradations and environment changes

Define term for target audience



Breakout: Drift

Scenario: *Target inventory management, predicting how much inventory should be in stock next week*

As a group, post and follow up in #lecture, tagging group members and @mlipbot:

- *Concept Drift:*
- *Data Drift:*
- *Schema Drift:*

Indicators of Concept Drift



Indicators of Concept Drift

Model degradations observed with telemetry

Telemetry indicates different outputs over time for similar inputs

Differences in influential features and feature importance over time

Relabeling training data changes labels

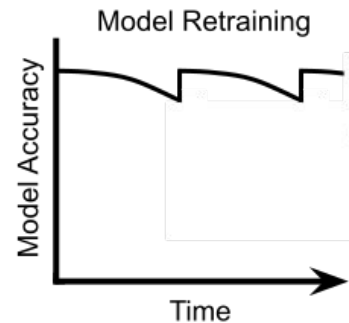
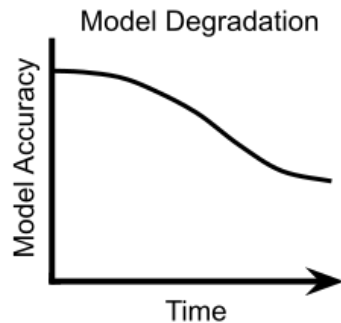
Interpretable ML models indicate rules that no longer fit

(many papers on this topic, typically on statistical detection)

Indicators of Data Drift



Indicators of Data Drift



Model degradations observed with telemetry

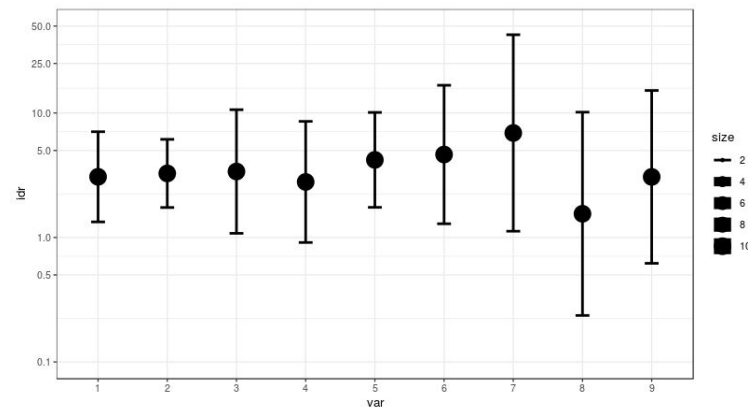
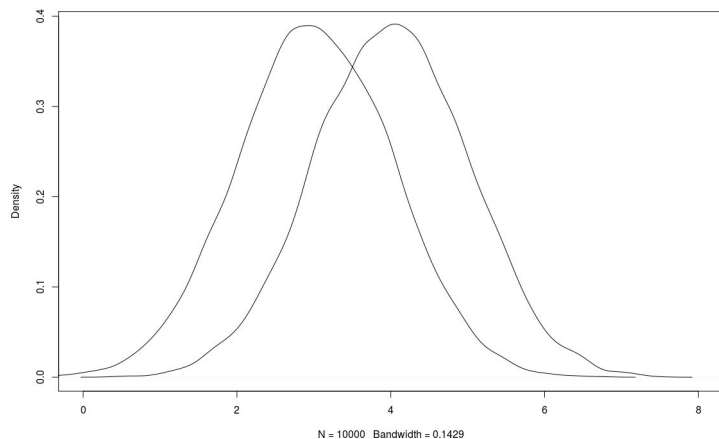
Distance between input distribution and training distribution increases

Average confidence of model predictions declines

Relabeling of training data retains stable labels

Detecting Data Drift

- Compare distributions over time (e.g., t-test)
- Detect both sudden jumps and gradual changes
- Distributions can be manually specified or learned (see invariant detection)



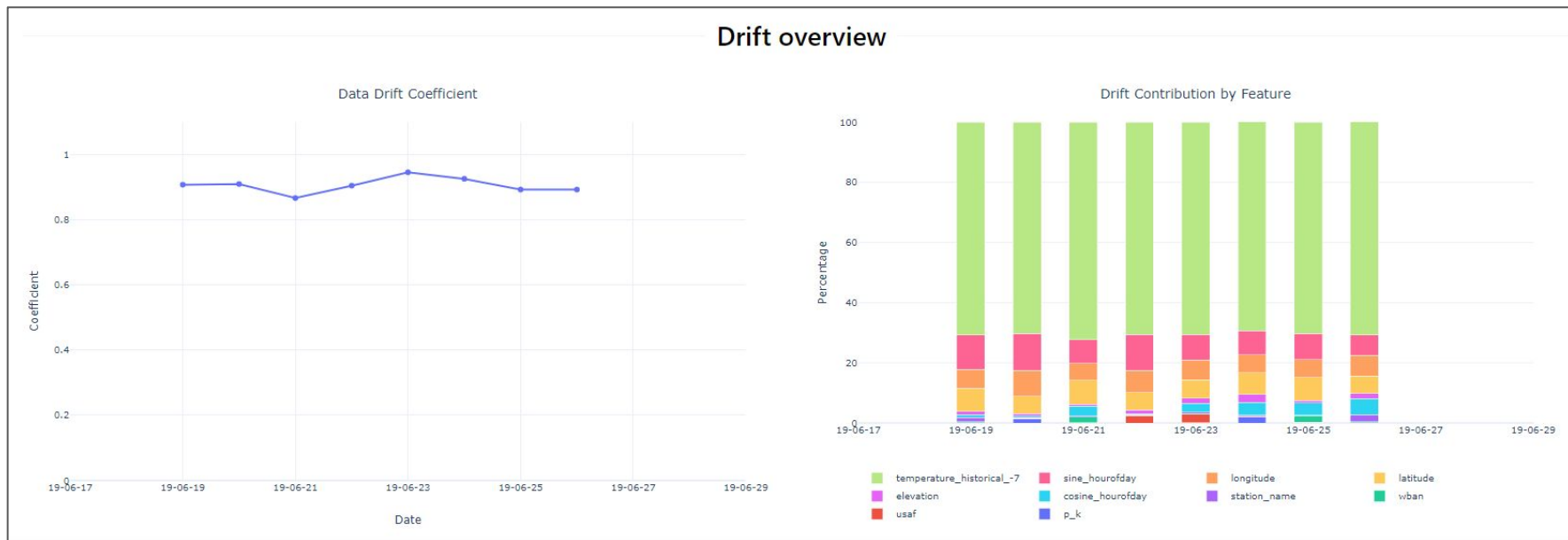
Drift Detection Tools

Many ML monitoring platforms support drift detection

- Evidently AI, NannyML, Alibi Detect, ...
- Arize AI, Fiddler AI, Whylabs, ...
- SageMaker Model Monitor, Azure ML Data Drift, Vertex Model Monitoring, ...

Possible to build your own...

Microsoft Azure Data Drift Dashboard



Dealing with Drift

Regularly retrain model on recent data

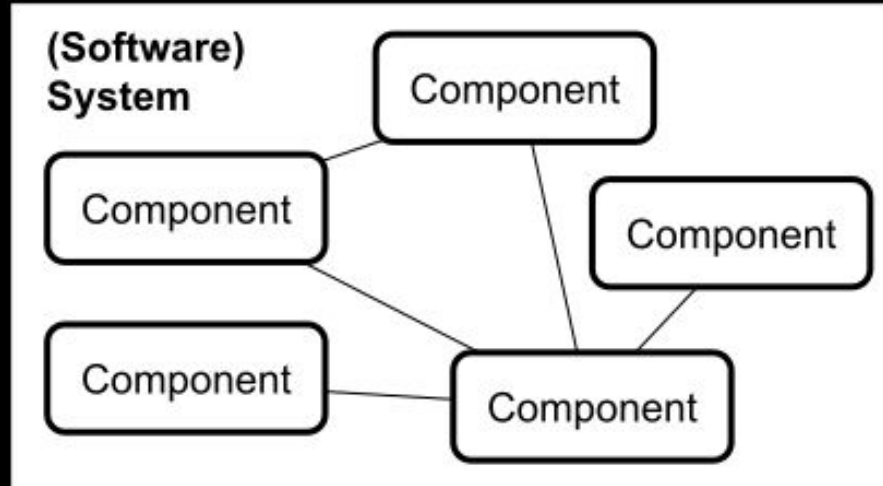
- Use evaluation in production to detect decaying model performance

Involve humans when increasing inconsistencies detected

- Monitoring thresholds, automation

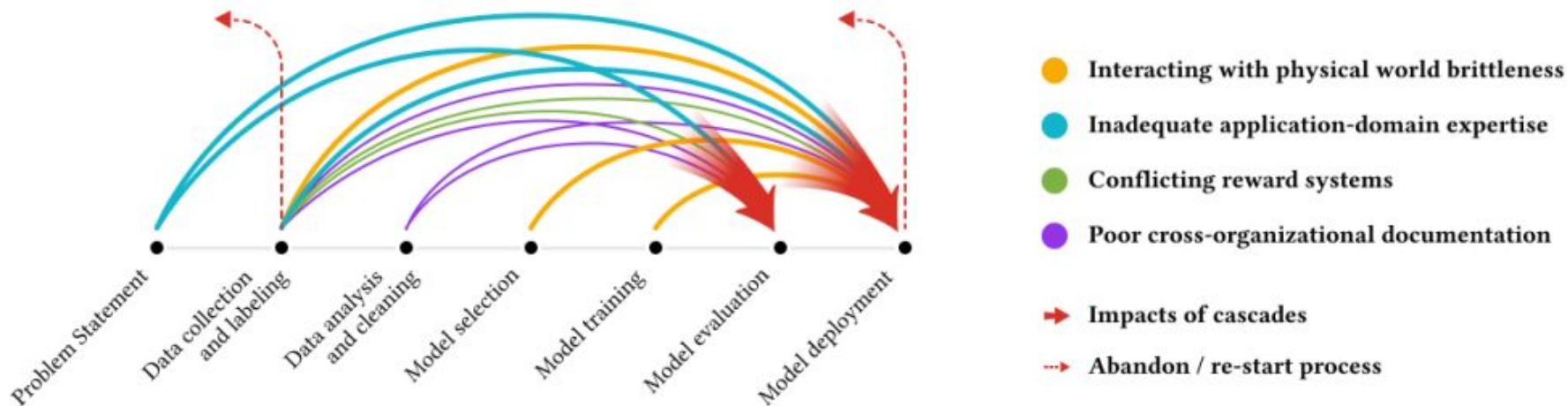
Monitoring, monitoring, monitoring!

Data Quality is a System-Wide Concern



Data Cascades

"Compounding events causing negative, downstream effects from data issues, that result in technical debt over time."



Common Data Cascades

Physical world brittleness

- Idealized data, ignoring realities and change of real-world data
- Static data, one time learning mindset, no planning for evolution

Inadequate domain expertise

- Not understand data and its context
- Involving experts only late for troubleshooting

Conflicting reward systems

- Missing incentives for data quality
- Not recognizing data quality importance, discard as technicality
- Missing data literacy with partners

Poor (cross-org.) documentation

- Conflicts at team/organization boundary
- Undetected drift

Sambasivan, N., et al. (2021). ["Everyone wants to do the model work, not the data work": Data Cascades in High-Stakes AI.](#) In Proc. Conference on Human Factors in Computing Systems.

Interacting with physical world brittleness

Brittle deployments interacting with not-digitised physical worlds

- **Time to manifest:** 2-3 years to emerge, almost always in the production stage
- **Impact:** Complete model failure, abandonment of projects, harms to beneficiaries from mispredictions
- **Triggers:** drifts
 - Hardware drifts (Rain, wind, fingerprints, shadows)
 - Environmental drifts (lighting, temperature, humidity)
 - Social drifts (new regulations, new user behaviors)
- **Address:** Monitor data source, retrain models, introduce noise in data

e.g. an AI model for the COVID-19 pandemic on day 1 versus day 100 required a total change in various assumptions since the pandemic and human responses were volatile and dynamic

Example for Inventory Mgmt Scenario?

Inadequate application-domain expertise

AI practitioners are responsible for data sense-making in contexts in which they do not have domain expertise.

- **Time to manifest:** After building models through client feedback & system performance
- **Impact:** Costly modification (improve labels, collect more data), Unanticipated downstream impacts
- **Triggers:**
 - Subjectivity in ground truths (e.g. Decision history on claims of insurance companies)
 - Poor application-domain expertise in finding representative data (e.g. Cannot take 90% of the data from one hospital and generalise for the entire world!)
- **Address:** Faithfully document data sources, involve domain experts in data collection

Example for Inventory Mgmt Scenario?

Conflicting Reward Systems

Misaligned incentives and priorities between practitioners, domain experts, and field partners.

- **Time to manifest:** Model deployment
- **Impact:** Costly iterations, moving to an alternate data source, quitting the project
- **Triggers:** Need annotation but...
 - Inserted as extraneous work
 - Not compensated well
 - Competing priority with partners' primary responsibility
- **Address:** Provide incentives & training

e.g., when a clinician spends a lot of time punching in data, not paying attention to the patient, that has a human cost

Example for Inventory Mgmt Scenario?

Poor Cross-organizational Documentation

Lack of documentation across various cross organisational relations, causing lack of understanding on metadata

- **Time to manifest:** manual reviews, by "chance"
- **Impact:** Wasted time and effort from using incorrect data, being blocked on building models, and discarding subsets or entire datasets
- **Triggers:**
 - inherited datasets lacked critical details
 - field partners not being aware of constraints in achieving good quality AI
- **Address:** Create a data curation plan in advance and take ample field notes in order to create reproducible assets for data

e.g., a lack of metadata and collaborators changing schema without understanding context led to a loss of four months of precious medical robotics data collection."

Example for Inventory Mgmt Scenario?

Discussion: Possible Data Cascades?

- Interacting with physical world brittleness
- Inadequate domain expertise
- Conflicting reward systems
- Poor (cross-organizational) documentation



Data Documentation

Let's use data documentation as an entry point to discuss what aspects of data we care about.

Data Quality is a System-Wide Concern

Data flows across components, e.g., from user interface into database to crowd-sourced labeling team into ML pipeline

Humans interacting with the system

- Entering data, labeling data
- Observed with sensors/telemetry
- Incentives, power structures, recognition

Organizational practices

- Value, attention, and resources given to data quality

Documentation at the interfaces is important

Data Quality Documentation

Teams rarely document expectations of data quantity or quality.

Data quality tests are rare, but some teams adopt defensive monitoring.

- Local tests about assumed structure and distribution of data
- Identify drift early and reach out to producing teams

Several ideas for documenting distributions, including [Datasheets](#) and [Dataset Nutrition Label](#)

- Mostly focused on static datasets, describing origin, consideration, labeling procedure, and distributions; [Example](#)

Data Card

Data Cards are for fostering transparent, purposeful and human-centered documentation of datasets within the practical contexts of industry and research. They are structured summaries of essential facts about various aspects of ML datasets...provide explanations of processes and rationales that shape the data and consequently the models.

Explore our Data Card template

This Data Card template captures 15 themes that we frequently look for when making decisions — many of which are not traditionally captured in technical dataset documentation.

Click on a theme below to see it in the Data Card and learn more:

Summary

Dataset Name (Acronym)

Write a short summary describing your dataset (limit 200 words). Include information about the content and topic of the data, sources and motivations for the dataset, benefits and the problems or use cases it is suitable for.

[DATASET LINK](#)

Dataset Link

[DATA CARD AUTHOR\(S\)](#)

- Name, Team: (Owner / Contributor / Manager)
- Name, Team: (Owner / Contributor / Manager)
- Name, Team: (Owner / Contributor / Manager)

Authorship ⓘ

[Publishers](#)

Open Images Extended - More Inclusively Annotated People (MIAP)

[Dataset Download](#)  • [Related Publication](#) 

This dataset was created for fairness research and fairness evaluations in person detection. This dataset contains 100,000 images sampled from Open Images V6 with additional annotations added. Annotations include the image coordinates of bounding boxes for each visible person. Each box is annotated with attributes for perceived gender presentation and age range presentation. It can be used in conjunction with Open Images V6.

Authorship

PUBLISHER(S)

Google LLC

INDUSTRY TYPE

Corporate - Tech

DATASET AUTHORS

Candice Schumann, Google, 2021
Susanna Ricco, Google, 2021
Utsav Prabhu, Google, 2021
Vittorio Ferrari, Google, 2021
Caroline Pantofaru, Google, 2021

FUNDING

Google LLC

FUNDING TYPE

Private Funding

DATASET CONTACT

open-images-extended@google.com

Motivations

DATASET PURPOSE(S)

Research Purposes

Machine Learning

Training, testing, and validation

KEY APPLICATION(S)

Machine Learning

Object Recognition

Machine Learning Fairness

PROBLEM SPACE

This dataset was created for fairness research and fairness evaluation with respect to person detection.

[See accompanying article](#) 

PRIMARY MOTIVATION(S)

- Provide more complete ground-truth for bounding boxes around people.
- Provide a standard fairness evaluation set for the broader fairness community.

INTENDED AND/OR SUITABLE USE CASE(S)

- **ML Model Evaluation for:** Person detection, Fairness evaluation
- **ML Model Training for:** Person detection, Object detection

Additionally:

- **Person detection:** Without specifying gender or age presentations
- **Fairness evaluations:** Over gender and age presentations
- **Fairness research:** Without building gender presentation or age classifiers

The Dataset Creator and Purpose

How to Use the Dataset

Use of Dataset

SAFETY OF USE

Conditional Use

There are some known unsafe applications.

UNSAFE APPLICATION(S)



Gender classification

Age classification

UNSAFE USE CASE(S)

This dataset **should not** be used to create gender or age classifiers. The intention of perceived gender and age labels is to capture gender and age presentation as assessed by a third party based on visual cues alone, rather than an individual's self-identified gender or actual age.

CONJUNCTIONAL USE

Safe to use with other datasets

KNOWN CONJUNCTIONAL DATASET(S)

- The data in this dataset can be combined with [Open Images V6](#)

KNOWN CONJUNCTIONAL USES

Analyzing bounding box annotations not annotated under the Open Images V6 procedure.

METHOD

Object Detection

SUMMARY

A person object detector can be trained using the Object Detection API in Tensorflow.

KNOWN CAVEATS

If this dataset is used in conjunction with the original Open Images dataset, negative examples of people should only be pulled from images with an explicit negative person image level label.

The dataset does not contain any examples not annotated as containing at least one person by the original Open Images annotation procedure.

METHOD

Fairness Evalutaion

SUMMARY

Fairness evaluations can be run over the splits of gender presentation and age presentation.

KNOWN CAVEATS

There still exists a gender presentation skew towards unknown and predominantly masculine, as well as an age presentation range skew towards middle.

Data fields

Dataset Snapshot

PRIMARY DATA TYPE(S)

Non-Sensitive Public Data
about people

DATASET SNAPSHOT

Total Instances	100,000
Training	70,000
Validation	7,410
Testing	22,590
Total boxes	454,331
Total labels	908,662
Average labels per image	9.08
Human annotated labels	All

DESCRIPTION OF CONTENT

Bounding boxes of people with perceived gender presentation attributes (*predominantly feminine, predominantly masculine, unknown*) and age range presentation attributes (*young, middle, older, unknown*). This adds adds nearly 100,000 new boxes that were not annotated under the original labeling pipeline of the core Open Images Dataset.

Note: All annotated images included at least one person bounding box in Open Images v6. 30,474 of the 100k images contain a MIAP-annotated bounding box with no corresponding annotation in Open Images. Almost 100,000 of the bounding boxes have no corresponding annotation in Open Images. Attributes were annotated for all boxes.

PRIMARY DATA MODALITY

Labels or Annotations

KNOWN CORRELATION(S)

- Gender presentation numbers are skewed towards predominantly perceived as **masculine & unknown**
- Age range presentation range numbers are skewed towards **middle**
- Perceived gender presentation is **unknown** for all bounding boxes with age range attribute annotated **young**

HOW TO INTERPRET A DATAPoint

Each datapoint includes a bounding box denoted by XMin, XMax, YMin, and YMax in normalized image coordinates. The next five attributes (IsOccluded through IsInsideOf) follow the [definitions from Open Images V6](#).

The **last two values** for each datapoint correspond to the gender presentation attribute and an age range presentation attribute, respectively.

Each annotation is linked to an Open Images key pointing to an image that can be found in [Common Visual Data Foundation \(CVDF\) repository](#).

Datapoint Example

EXAMPLE OF ACTUAL DATA POINT WITH DESCRIPTIONS

Field	Value	Description
ImageID	164b0e6d1fcf8e61	The image this box lives in
LabelName	/m/01g317	Labels are identified by MIDs (Machine-generated Ids) as can be found in Freebase or Google Knowledge Graph API . Label descriptions here
Confidence	1	A dummy value, always 1
XMin	0.897112	Normalized image coordinates indicating the leftmost pixel of the annotation
XMax	0.987365	Normalized image coordinates indicating the rightmost pixel of the annotation
YMin	0.615523	Normalized image coordinates indicating the topmost pixel of the annotation
YMax	0.895307	Normalized image coordinates indicating the bottommost pixel of the annotation
IsOccluded	0	Binary value indicating if the object is occluded by another object in the image
IsTruncated	1	Binary value indicating if the object extends beyond the boundary of the image
IsGroupOf	0	Binary value indicating if the box spans a group of objects
IsDepictionOf	1	Binary value indicating if the object is a depiction and not a real physical instance
IsInsideOf	1	Binary value indicating if the image is taken from the inside of the object
IsInsideOf	1	Binary value indicating if the image is taken from the inside of the object
GenderPresentation	Predominantly Masculine	Indicates the perceived gender presentation of the subject assessed by a third party
AgePresentation	Middle	Indicates the perceived age range of the subject assessed by a third party

Data source and collection

Data Collection

DATA COLLECTION METHOD(S)

Derived

Vendor Collection Efforts

DATA SOURCES BY COLLECTION METHOD(S)

Images	Open Images V6
Labels	Human annotators
Bounding Boxes	Human annotators

EXCLUDED DATA

No excluded data

SUMMARIES OF DATA COLLECTION METHODS

100,000 images randomly sampled from the positive set of Open Images V6, which contains approximately 9.9M images

- Training Set: 70,000 sampled from 9,011,219 images
- Testing/Validation: 30,000 sampled from 167,056 images

DATA SELECTION CRITERIA - SCRAPING

- Images were sampled from the positive subset of training and testing/validation containing annotator-verified image labels
- Images contained at least one of five person classes (**man**, **woman**, **boy**, **girl**, or **person**)

Note: We did not include non-binary as a class label as it is not possible to label gender identity from images. Gender identity should only be used in situations where participants are able to self-report gender.

Labelling Process

METHOD(S)

Human labels

LABEL TYPE(S)

Human Attributes Labels	
PerceivedGender	Human annotators
PercievedAge	
Bounding Boxes (where missing)	
rectangular box	Drawn by human annotators, computed into normalized image coordinates
IsTruncated	Object attributes annotated by human annotators to describe the bounding box
IsOccluded	
IsGroup	
IsInside	
IsDepiction	

LABEL TYPE

Bounding Box

LABEL DISTRIBUTION

Label	Original	MIAP
boxes	357,870	454,331

Above: Counts of boxes across the MIAP in comparison to the 100,000 samples from Open Images V6. For a more detailed breakdown, see our paper.

METHOD(S) SUMMARY

Compensated workers based out of India were recruited to annotate and re-label images. Bounding boxes were created in an image and perceived age ranges as well as perceived presentation were labeled.

Labeling Process

LABEL DESCRIPTION(S)

Bounding Box: A rectangular bounding box around each person in an image. Object Attributes include: is truncated, is occluded, is inside, is group, and is depiction.

LABELING TASK(S) OR PROCEDURE(S)

“Create the bounding box around all people”
“Label object attributes”

Annotators were asked to place boxes around all people in an image. If there were 5 or more people grouped together a single box was used and a group of attribute was associated with that box. Annotators were asked if the person inside of the box was truncated, occluded, or inside of something. They were also asked if the person inside of the box was a depiction of a person (such as a painting or figurine).

Labelling Process

LABEL TYPE	LABEL DISTRIBUTION	LABEL DESCRIPTION(S)															
Perceived Gender	<table><tr><th>Label</th><th>Original</th><th>MIAP</th></tr><tr><td>Predominantly feminine</td><td>76,283</td><td>100,672</td></tr><tr><td>Predominantly masculine</td><td>143,320</td><td>174,047</td></tr><tr><td>Unknown gender presentation</td><td>138,267</td><td>179,612</td></tr></table> Above: Counts of boxes for different classes of the perceived gender label across the MIAP in comparison to the 100,000 samples from Open Images V6. For a more detailed breakdown, see our paper.	Label	Original	MIAP	Predominantly feminine	76,283	100,672	Predominantly masculine	143,320	174,047	Unknown gender presentation	138,267	179,612	Classes for the perceived gender presentation label are: • predominantly feminine • predominantly masculine • unknown LABELING TASK(S) OR PROCEDURE(S) “Label the perceived gender presentation” Annotators were asked to select either predominantly feminine, predominantly masculine, or unknown to describe the human-perceived gender presentation of an individual based on the visual cues in the image. Note: Gender presentation for people marked as young is always set to unknown.			
Label	Original	MIAP															
Predominantly feminine	76,283	100,672															
Predominantly masculine	143,320	174,047															
Unknown gender presentation	138,267	179,612															
LABEL TYPE	LABEL DISTRIBUTION	LABEL DESCRIPTION(S)															
Perceived Age	<table><tr><th>Label</th><th>Original</th><th>MIAP</th></tr><tr><td>young</td><td>21,548</td><td>28,806</td></tr><tr><td>middle</td><td>198,055</td><td>233,674</td></tr><tr><td>older</td><td>no such label</td><td>9,023</td></tr><tr><td>Unknown</td><td>138,267</td><td>182,828</td></tr></table> Above: Counts of boxes for different classes of the perceived age label across the MIAP in comparison to the 100,000 samples from Open Images V6. For a more detailed breakdown, see our paper.	Label	Original	MIAP	young	21,548	28,806	middle	198,055	233,674	older	no such label	9,023	Unknown	138,267	182,828	Classes for the perceived age range label are: • young • middle • older • unknown LABELING TASK(S) OR PROCEDURE(S) “Label the perceived age range” Annotators were asked to select either either young, middle, older, or unknown to describe the perceived age range of an individual based on their appearance in the image. Annotators were instructed to prefer the older of two categories in situations where there was enough information to form an impression but were unsure of a boundary case. <i>For example</i>, someone who appears old enough to possibly belong to middle should be assigned that attribute label.
Label	Original	MIAP															
young	21,548	28,806															
middle	198,055	233,674															
older	no such label	9,023															
Unknown	138,267	182,828															

Entries in data card

Very good reference for data quality, but way too difficult to fill out for every dataset so usually ignored...

Table 2: Content themes in the Data Card template. Our content schema extends the constitution of traditional dataset documentation to include explanations, rationales, and instructions pertaining to 31 themes. We anticipate that not all themes will be uniformly relevant to all datasets or equally applicable to features within a single dataset.

(1) The publishers of the dataset and access to them	(17) The data collection process (inclusion, exclusion, filtering criteria)
(2) The funding of the dataset	(18) How the data was cleaned, parsed, and processed (transformations, sampling, etc.)
(3) The access restrictions and policies of the dataset	(19) Data rating in the dataset, process, description and/or impact
(4) The wipeout and retention policies of the dataset	(20) Data labeling in the dataset, process, description and/or impact
(5) The updates, versions, refreshes, additions to the data of the dataset	(21) Data validation in the dataset, process, description and/or impact
(6) Detailed breakdowns of features of the dataset	(22) The past usage and associated performance of the dataset (eg. models trained)
(7) Details about collected attributes which are absent from the dataset or the dataset's documentation	(23) Adjudication policies and processes related to the dataset (labeler instructions, inter-rater policy, etc.)
(8) The original upstream sources of the data	(24) Relevant associated regulatory or compliance policies (GDPR, licenses, etc.)
(9) The nature (data modality, domain, format, etc.) of the dataset	(25) Dataset Infrastructure and/or pipeline implementation
(10) What typical and outlier examples in the dataset look like	(26) Descriptive statistics of the dataset (mean, standard deviations, etc.)
(11) Explanations and motivations for creating the dataset	(27) Any known patterns (correlations, biases, skews) within the dataset
(12) The intended applications of the dataset	(28) Human attributes (socio-cultural, geopolitical, or economic representation)
(13) The safety of using the dataset in practice (risks, limitations, and trade-offs)	(29) Fairness-related evaluations and considerations of the dataset
(14) Expectations around using the dataset with other datasets or tables (feature engineering, joining, etc.)	(30) Definitions and explanations for technical terms used in the Data Card (metrics, industry-specific terms, acronyms)
(15) The maintenance status and version of the dataset	(31) Domain-specific knowledge required to use the dataset
(16) Difference across previous and current versions of the dataset	

Data Cards give you an idea of what might impact data quality

We will touch on some:

- Data quality, in terms of noise (e.g. from upstream data source, from human labelers)
- Data drifts (static dataset and its source, vs. the world now)
- Data quality, in terms of distributions (disagreements between annotators, biases towards certain distribution, annotation incentives – tend to select easy labels, etc.)
- Data curation (shape the data towards what you want)

Preview: Inaccurate Data can also be caused by factors other than drift

How do you detect and fix more systemic data quality issues?

Challenge from collection and processing: Bias

The concept of "raw data" might be misleading; Data is always some proxy we actively collect to represent the world. Someone decides...

- What data to collect
- How to collect them
- What scale to use

They change what you can do with the data

Shipment Received Note

Goods Received Receipt

Supplier: Mr. Adam Smith		Destination: Marlon De compo			
Shipment Date	June 17, 2015	Receipt No.	09XC10	Genre: Shipment Delivery Receipt	
Order Number	67-NY25	Delivery Duration	5 days		
Goods Details	Packing Weight	Packing Size	Quantity	Delivered Quantity	Comments

Received by: Mr. Ronald Richard Olinde

Recommended Reading: Gitelman, Lisa, Virginia Jackson, Daniel Rosenberg, Travis D. Williams, Kevin R. Brine, Mary Poovey, Matthew Stanley et al. ["Data bite man: The work of sustaining a long-term study."](#) In "Raw Data" Is an Oxymoron, (2013), MIT Press: 147-166.

What happens: Data is inaccurate (for what you want to do)

Missing data

Biased data

Systematic errors in data distribution

Summary

Data quality is a system-level concern

- Data quality at the interface between components
- Documentation and monitoring often poor
- Involves organizational structures, incentives, ethics, ...

Data from many sources, often inaccurate, imprecise, inconsistent, incomplete, ...
– many different forms of data quality problems

Many mechanisms for enforcing consistency and cleaning

- Data schema ensures format consistency
- Data quality rules ensure invariants across data points

Concept and data drift are key challenges → monitor

Further Readings

- Schelter, S., Lange, D., Schmidt, P., Celikel, M., Biessmann, F. and Grafberger, A., 2018. Automating large-scale data quality verification. Proceedings of the VLDB Endowment, 11(12), pp.1781-1794.
- Polyzotis, Neoklis, Martin Zinkevich, Sudip Roy, Eric Breck, and Steven Whang. "Data validation for machine learning." Proceedings of Machine Learning and Systems 1 (2019): 334-347.
- Polyzotis, Neoklis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. 2017. "Data Management Challenges in Production Machine Learning." In Proceedings of the 2017 ACM International Conference on Management of Data, 1723–26. ACM.
- Theo Rekatsinas, Ihab Ilyas, and Chris Ré, "HoloClean - Weakly Supervised Data Repairing." Blog, 2017.
- Ilyas, Ihab F., and Xu Chu. Data cleaning. Morgan & Claypool, 2019.
- Moreno-Torres, Jose G., Troy Raeder, Rocío Alaiz-Rodríguez, Nitesh V. Chawla, and Francisco Herrera. "A unifying view on dataset shift in classification." Pattern recognition 45, no. 1 (2012): 521-530.
- Vogelsang, Andreas, and Markus Borg. "Requirements Engineering for Machine Learning: Perspectives from Data Scientists." In Proc. of the 6th International Workshop on Artificial Intelligence for Requirements Engineering (AIRE), 2019.
- Humbatova, Nargiz, Gunel Jahangirova, Gabriele Bavota, Vincenzo Riccio, Andrea Stocco, and Paolo Tonella. "Taxonomy of real faults in deep learning systems." In Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, pp. 1110-1121. 2020.